MLflow Model Patterns

> **Document Owner:** ML Engineering | **Status:** Approved | **Last Updated:** February 2026

Overview

Implementation patterns for the full ML model lifecycle on Databricks: feature engineering, experiment tracking, model training, registration, promotion, serving, and monitoring. All models are governed by Unity Catalog and tracked via MLflow.

For GenAI agent patterns, see 51-genai-agent-patterns.md. For GenAI governance standards, see 52-genai-standards.md.

Golden Rules

ID	Rule	Severity	WAF Pillar
ML-01	Feature tables in Unity Catalog with primary keys	Critical	Governance
ML-02	`output_schema` required for UC models	Critical	Governance
ML-03	NaN/Inf handling at feature table creation, not training	Critical	Governance
ML-04	Organize experiments by lifecycle stage	Required	Governance
ML-05	Label binarization for classifiers (0/1 integers)	Critical	Governance
ML-06	Exclude label and key columns from feature set	Critical	Governance
ML-07	Register all models in Unity Catalog with aliases	Critical	Governance
ML-08	Use Champion/Challenger pattern for model promotion	Required	Governance
ML-09	Log parameters, metrics, and artifacts for every run	Required	Governance

---

Platform & Enterprise Prerequisites

> This document defines **solution patterns**. The following EA/PA rules are prerequisites.

Prerequisite	Rule IDs	Document
Unity Catalog tables	DL-01	12-unity-catalog-tables
UC governance for AI assets	GOV-06	15-unity-catalog-governance
Data modeling standards	DM-09	06-data-modeling

ML-01: Feature Tables in Unity Catalog

Rule

All feature tables must be created in Unity Catalog using the Feature Engineering Client, with defined primary keys and optional timestamp keys for point-in-time lookups.

Why It Matters

- Enables automatic feature lookup during training and inference
- `fe.score_batch()` joins features at inference time using primary keys
- Provides lineage from features back to source tables
- Supports feature sharing across teams and models

Implementation

```
from databricks.feature_engineering import FeatureEngineeringClient,
FeatureLookup

fe = FeatureEngineeringClient()

# Create feature table from Gold layer
feature_df = spark.sql("""
    SELECT
        customer_id,                    -- Primary key
        lifetime_value,
        order_count,
        avg_order_value,
        days_since_last_order,
        customer_segment
    FROM gold.dim_customer
    WHERE is_current = true
""")

fe.create_table(
    name=f"{catalog}.{schema}.customer_features",
    primary_keys=["customer_id"],
    df=feature_df,
    description="Customer features derived from gold.dim_customer. Updated
daily."
)
```

With timestamp key (point-in-time correctness):

```
fe.create_table(
    name=f"{catalog}.{schema}.customer_features_ts",
    primary_keys=["customer_id"],
    timestamp_keys=["feature_date"],      # Enables point-in-time lookup
    df=feature_df_with_dates,
    description="Time-series customer features for temporal ML training."
)
```

Feature Lookup for Training

```
# Define which features to use from which tables
feature_lookups = [
    FeatureLookup(
        table_name=f"{catalog}.{schema}.customer_features",
        feature_names=["lifetime_value", "order_count",
"avg_order_value"],
        lookup_key="customer_id"
    ),
    FeatureLookup(
        table_name=f"{catalog}.{schema}.product_features",
        feature_names=["category_popularity", "return_rate"],
        lookup_key="product_id"
    )
]

# Create training set - features joined automatically
training_set = fe.create_training_set(
    df=labels_df,                          # Must contain primary keys +
label
    feature_lookups=feature_lookups,
    label="label",
    exclude_columns=["customer_id", "product_id"]  # Keys excluded from
features
)

training_df = training_set.load_df()
```

Common Mistakes

Mistake	Why It Fails	Fix
No primary keys	`fe.score_batch()` can't join features	Always define `primary_keys`
Features in a non-UC table	No governance, lineage, or sharing	Use `fe.create_table()`
Manual joins at inference	Skew between training and serving	Use `FeatureLookup`

ML-02: Output Schema Required

Rule

All models registered in Unity Catalog must include an explicit `output_schema` to define the prediction output type.

Why It Matters

- Unity Catalog requires schema metadata for model governance
- Enables downstream consumers to understand prediction output without running the model

- Required for `fe.score_batch()` to work correctly
- Supports schema validation in serving endpoints

Implementation

```
from mlflow.models.signature import ColSpec, Schema, DataType

# Define based on model type
output_schema = Schema([ColSpec(DataType.double)])   # Regression
# output_schema = Schema([ColSpec(DataType.long)])   # Classification
# output_schema = Schema([ColSpec(DataType.string)]) # Text output

fe.log_model(
    model=trained_model,
    artifact_path="model",
    flavor=mlflow.sklearn,
    output_schema=output_schema,
    training_set=training_set,                        # Links features
    registered_model_name=f"{catalog}.{schema}.{model_name}"
)
```

Model Type	DataType	Example
Regression	`DataType.double`	Price prediction
Binary classification	`DataType.long`	Churn (0/1)
Multi-class classification	`DataType.long`	Segment (0/1/2/3)
Probability output	`DataType.double`	Churn probability (0.0-1.0)
Anomaly detection	`DataType.long`	Anomaly flag (0/1)

ML-03: NaN Handling at Source

Rule

Clean NaN and Inf values at feature table creation time, not during training preprocessing. Training preprocessing that fills NaN is NOT applied during `fe.score_batch()`.

Why It Matters

- `fe.score_batch()` bypasses your training preprocessing pipeline
- Raw feature values are passed directly to the model
- If NaN exists in the feature table, the model receives NaN at inference
- Results in `Input X contains NaN` errors in production

Implementation

```python
from pyspark.sql import functions as F

def clean_numeric_features(df, numeric_columns):
    """Clean NaN/Inf at source - critical for fe.score_batch()
compatibility."""
    for col_name in numeric_columns:
        df = df.withColumn(
            col_name,
            F.when(
                F.col(col_name).isNull() | F.isnan(F.col(col_name)),
                F.lit(0.0)
            ).when(
                F.col(col_name) == float('inf'),
                F.lit(0.0)
            ).when(
                F.col(col_name) == float('-inf'),
                F.lit(0.0)
            ).otherwise(F.col(col_name))
        )
    return df

# Apply BEFORE writing to feature table
numeric_cols = ["lifetime_value", "order_count", "avg_order_value"]
clean_df = clean_numeric_features(feature_df, numeric_cols)
fe.write_table(name=f"{catalog}.{schema}.customer_features", df=clean_df,
mode="merge")
```

Anti-Pattern

```python
# ❌ WRONG: This only works during training, NOT during fe.score_batch()
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
X_train = imputer.fit_transform(X_train)  # Fills NaN for training only!
```

ML-04: Experiment Organization

Rule

Organize MLflow experiments by lifecycle stage with consistent naming. Separate development iteration from formal evaluation and production deployment.

Why It Matters

- Prevents development noise from polluting production experiment history
- Enables team members to find relevant runs by stage
- Supports audit trail for model promotion decisions
- Simplifies cleanup of experimental runs

Implementation

Stage	Experiment Path	Purpose	Retention
Development	`/Shared/{project}_ml_{model}_development`	Rapid iteration, hyperparameter search	90 days
Evaluation	`/Shared/{project}_ml_{model}_evaluation`	Formal model comparison, selection	1 year
Production	`/Shared/{project}_ml_{model}_production`	Production training runs only	Permanent

```
import mlflow

# Set experiment for the current stage
mlflow.set_experiment(f"/Shared/customer360_ml_churn_development")

with mlflow.start_run(run_name="xgb_v3_depth8") as run:
    # Log parameters
    mlflow.log_params({
        "model_type": "xgboost",
        "max_depth": 8,
        "learning_rate": 0.1,
        "n_estimators": 500,
        "feature_table": f"{catalog}.{schema}.customer_features",
        "label_threshold": 0.10
    })

    # Train model
    model = xgb.XGBClassifier(max_depth=8, learning_rate=0.1,
n_estimators=500)
    model.fit(X_train, y_train)

    # Log metrics
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    mlflow.log_metrics({
        "accuracy": accuracy_score(y_test, y_pred),
        "f1_score": f1_score(y_test, y_pred),
        "auc_roc": roc_auc_score(y_test, y_proba),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "train_size": len(X_train),
        "test_size": len(X_test)
    })

    # Log artifacts
    mlflow.log_dict(
```

```
        {"features": feature_cols, "label": "label"},
        "feature_config.json"
    )
```

ML-05: Label Binarization

Rule

Classification labels must be integer-encoded (0/1 for binary, 0/1/.../N for multi-class). Never pass continuous floats as classification labels.

Why It Matters

- XGBoost, LightGBM, and other tree classifiers expect integer labels
- Continuous labels cause `base_score must be in (0,1)` errors
- Evaluation metrics (accuracy, F1, AUC) require discrete labels
- Threshold selection should be explicit, not implicit

Implementation

```
from pyspark.sql import functions as F

# Binary classification - explicit threshold
labeled_df = df.withColumn(
    "label",
    F.when(F.col("churn_rate") > 0.10, 1).otherwise(0).cast("integer")
)

# Verify label distribution before training
label_counts = labeled_df.groupBy("label").count().collect()
label_dist = {row["label"]: row["count"] for row in label_counts}

if len(label_dist) < 2:
    raise ValueError(f"Single class detected: {label_dist}. Cannot train
classifier.")

minority_ratio = min(label_dist.values()) / sum(label_dist.values())
if minority_ratio < 0.05:
    print(f"WARNING: Severe class imbalance ({minority_ratio:.1%}
minority). "
          "Consider SMOTE, class_weight, or scale_pos_weight.")
```

ML-06: Exclude Label and Keys from Features

Rule

Always exclude label columns, primary keys, and timestamp columns from the feature set before training. Use an explicit allowlist or blocklist pattern.

Why It Matters

- Including the label causes data leakage (model memorizes the answer)
- Including primary keys teaches the model to memorize IDs
- Including timestamps creates time-dependent features that don't generalize

Implementation

```
def get_feature_columns(df, exclude_columns):
    """Get feature columns by excluding labels, keys, and metadata."""
    return [c for c in df.columns if c not in exclude_columns]

# Explicit exclusion list
EXCLUDE = ["label", "customer_id", "product_id", "feature_date",
"processed_at"]
feature_cols = get_feature_columns(training_df, EXCLUDE)

# Verify no leakage
assert "label" not in feature_cols, "Label column in features!"
assert all(k not in feature_cols for k in ["customer_id", "product_id"]),
"Key column in features!"

X_train = training_df[feature_cols].values
y_train = training_df["label"].values
```

ML-07: Model Registration in Unity Catalog

Rule

All models must be registered in Unity Catalog with descriptive metadata. Use the three-level namespace: `catalog.schema.model_name`.

Why It Matters

- Centralized access control (GRANT/REVOKE on models)
- Version history with full lineage to training data and features
- Required for model serving endpoints
- Enables cross-team model discovery and reuse

Implementation

```
import mlflow

# Register during logging (preferred)
fe.log_model(
```

```
    model=trained_model,
    artifact_path="model",
    flavor=mlflow.sklearn,
    output_schema=output_schema,
    training_set=training_set,
    registered_model_name=f"{catalog}.ml.customer_churn_model"
)

# Or register an existing run
mlflow.register_model(
    model_uri=f"runs:/{run_id}/model",
    name=f"{catalog}.ml.customer_churn_model"
)
```

Naming Convention

```
{catalog}.ml.{domain}_{task}_model

Examples:
- prod_catalog.ml.customer_churn_model
- prod_catalog.ml.product_demand_forecast
- prod_catalog.ml.fraud_detection_model
```

Required Metadata

```
from mlflow import MlflowClient

client = MlflowClient()

# Set model description
client.update_registered_model(
    name=f"{catalog}.ml.customer_churn_model",
    description="Predicts customer churn probability. Trained on
customer_features table. "
                "Binary classification (0=retained, 1=churned). Refreshed
weekly."
)

# Set model tags
client.set_registered_model_tag(
    name=f"{catalog}.ml.customer_churn_model",
    key="domain", value="customer"
)
client.set_registered_model_tag(
    name=f"{catalog}.ml.customer_churn_model",
    key="team", value="ml-engineering"
)
client.set_registered_model_tag(
    name=f"{catalog}.ml.customer_churn_model",
```

```
    key="task_type", value="binary_classification"
)
```

ML-08: Champion/Challenger Model Promotion

Rule

Use MLflow model aliases (`Champion`, `Challenger`) for model promotion. Never reference models by version number in production code.

Why It Matters

- Decouples deployment from specific version numbers
- Enables safe rollback by reassigning the alias
- Supports A/B testing via Challenger alias
- Production code references `@Champion` and never needs updating

Promotion Workflow

```
Development → Evaluation → Challenger → Champion
                              ↑            ↑
                          Validate     Promote after
                          in staging   metrics pass
```

Implementation

```python
from mlflow import MlflowClient

client = MlflowClient()
model_name = f"{catalog}.ml.customer_churn_model"

# Step 1: Train and register new version (creates version N)
# ... (training code from above)

# Step 2: Set as Challenger for validation
client.set_registered_model_alias(
    name=model_name,
    alias="Challenger",
    version=new_version
)

# Step 3: Validate Challenger against Champion
champion_uri = f"models:/{model_name}@Champion"
challenger_uri = f"models:/{model_name}@Challenger"

champion_metrics = evaluate_model(champion_uri, test_df)
challenger_metrics = evaluate_model(challenger_uri, test_df)
```

```
# Step 4: Promote if metrics improve
if challenger_metrics["auc_roc"] > champion_metrics["auc_roc"]:
    client.set_registered_model_alias(
        name=model_name,
        alias="Champion",
        version=new_version
    )
    print(f"Promoted version {new_version} to Champion "
          f"(AUC: {champion_metrics['auc_roc']:.4f} →
{challenger_metrics['auc_roc']:.4f})")
else:
    print(f"Challenger did not improve. Champion retained.")
```

Promotion Gates

Gate	Criteria	Blocking
Metric improvement	AUC/F1 >= Champion	Yes
Data quality	No NaN/Inf in predictions	Yes
Latency	P95 inference < threshold	Yes (serving)
Bias check	Fairness metrics within tolerance	Required
Stakeholder review	Business approval for high-impact models	Required

---

ML-09: Experiment Tracking

Rule

Log parameters, metrics, and artifacts for every training run. Use consistent metric names across experiments to enable comparison.

Why It Matters

- Enables reproducibility of any past experiment
- Supports model comparison and selection decisions
- Creates audit trail for regulatory compliance
- Prevents "which notebook produced this model?" confusion

What to Log

Category	What	Example
Parameters	Hyperparameters, thresholds, feature config	`max_depth=8`, `label_threshold=0.10`
Metrics	Evaluation scores, dataset sizes	`auc_roc=0.89`, `train_size=50000`

Category	What	Example
Artifacts	Feature lists, confusion matrices, SHAP plots	`feature_config.json`, `confusion_matrix.png`
Tags	Run metadata	`model_type=xgboost`, `dataset_version=v3`
Input data	Feature table reference	MLflow dataset logging

Implementation

```
import mlflow

with mlflow.start_run(run_name="churn_xgb_v4") as run:
    # Parameters
    mlflow.log_params({
        "model_type": "xgboost",
        "max_depth": 8,
        "learning_rate": 0.1,
        "n_estimators": 500,
        "feature_table": f"{catalog}.{schema}.customer_features",
        "label_column": "label",
        "label_threshold": 0.10,
        "train_start_date": "2024-01-01",
        "train_end_date": "2025-12-31"
    })

    # Tags
    mlflow.set_tags({
        "model_type": "xgboost",
        "dataset_version": "v3",
        "triggered_by": "scheduled_retrain"
    })

    # Train
    model = xgb.XGBClassifier(**params)
    model.fit(X_train, y_train)

    # Metrics
    y_pred = model.predict(X_test)
    mlflow.log_metrics({
        "accuracy": accuracy_score(y_test, y_pred),
        "f1_score": f1_score(y_test, y_pred),
        "auc_roc": roc_auc_score(y_test, y_proba),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "train_rows": len(X_train),
        "test_rows": len(X_test),
        "feature_count": len(feature_cols)
    })
```

```
    # Feature importance artifact
    importance = dict(zip(feature_cols, model.feature_importances_))
    mlflow.log_dict(importance, "feature_importance.json")
```

Batch Inference Pattern

Standard Batch Scoring

```
def run_batch_inference(fe, catalog, schema, model_name):
    """Score all records using Champion model with feature auto-lookup."""
    model_uri = f"models:/{catalog}.ml.{model_name}@Champion"
    scoring_df = spark.table(f"{catalog}.{schema}.scoring_data")

    # fe.score_batch automatically joins features from the feature table
    predictions = fe.score_batch(model_uri=model_uri, df=scoring_df)

    # Write predictions to Gold layer for BI consumption
    (predictions
        .withColumn("scored_at", F.current_timestamp())
        .withColumn("model_version",
F.lit(get_champion_version(model_name)))
        .write
        .mode("overwrite")
        .saveAsTable(f"{catalog}.gold.{model_name}_predictions"))
```

Scheduled Retraining Pattern

```
# Asset Bundle: Scheduled weekly retrain + score
resources:
  jobs:
    ml_retrain_and_score:
      name: "[${bundle.target}] Customer Churn - Retrain & Score"

      schedule:
        quartz_cron_expression: "0 0 6 ? * MON"  # Weekly Monday 6 AM
        timezone_id: "UTC"

      tasks:
        - task_key: retrain
          notebook_task:
            notebook_path: ../src/ml/train_churn_model.py
            base_parameters:
              catalog: ${var.catalog}
              schema: ${var.ml_schema}

        - task_key: validate
          depends_on:
            - task_key: retrain
```

```
            notebook_task:
              notebook_path: ../src/ml/validate_challenger.py

          - task_key: score
            depends_on:
              - task_key: validate
            notebook_task:
              notebook_path: ../src/ml/batch_score.py

        tags:
          team: ml-engineering
          workload_type: ml-training
```

Real-Time Serving Pattern

Model Serving Endpoint

```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()

# Create or update serving endpoint
w.serving_endpoints.create(
    name="customer-churn-endpoint",
    config={
        "served_entities": [{
            "entity_name": f"{catalog}.ml.customer_churn_model",
            "entity_version": None,         # Uses Champion alias
            "workload_size": "Small",
            "scale_to_zero_enabled": True    # Cost optimization
        }],
        "auto_capture_config": {
            "catalog_name": catalog,
            "schema_name": "monitoring",
            "table_name_prefix": "churn_endpoint",
            "enabled": True                  # Payload logging for
monitoring
        }
    }
)
```

Querying the Endpoint

```
import requests

# From notebook or application
response = w.serving_endpoints.query(
    name="customer-churn-endpoint",
```

```
    dataframe_records=[{
        "customer_id": "CUST-00001234",
        "lifetime_value": 5420.50,
        "order_count": 47,
        "days_since_last_order": 15
    }]
)
```

Hyperparameter Tuning Pattern

```
from hyperopt import fmin, tpe, hp, STATUS_OK, Trials
import mlflow

def objective(params):
    """Hyperopt objective function with MLflow tracking."""
    with mlflow.start_run(nested=True):
        mlflow.log_params(params)

        model = xgb.XGBClassifier(
            max_depth=int(params["max_depth"]),
            learning_rate=params["learning_rate"],
            n_estimators=int(params["n_estimators"]),
            use_label_encoder=False,
            eval_metric="logloss"
        )
        model.fit(X_train, y_train)

        auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
        mlflow.log_metric("auc_roc", auc)

        return {"loss": -auc, "status": STATUS_OK}

search_space = {
    "max_depth": hp.quniform("max_depth", 3, 12, 1),
    "learning_rate": hp.loguniform("learning_rate", -5, 0),
    "n_estimators": hp.quniform("n_estimators", 100, 1000, 50)
}

with mlflow.start_run(run_name="hyperparam_search"):
    best = fmin(fn=objective, space=search_space, algo=tpe.suggest,
max_evals=50)
```

Validation Checklist

Feature Engineering

- [ ] Feature table created via `fe.create_table()` in Unity Catalog
- [ ] Primary keys defined

- [ ] Timestamp keys defined (if point-in-time needed)
- [ ] NaN/Inf cleaned at table creation (not training)
- [ ] Feature table description documented
- [ ] Features derived from Gold layer where possible (DM-09)

Training

- [ ] Labels are integer-encoded (0/1 for binary)
- [ ] Label excluded from feature columns
- [ ] Label distribution checked (class balance)
- [ ] `output_schema` defined and matches model output
- [ ] Parameters, metrics, and artifacts logged to MLflow
- [ ] Experiment organized by lifecycle stage
- [ ] `training_set` passed to `fe.log_model()` for lineage

Registration & Promotion

- [ ] Model registered in Unity Catalog (`catalog.schema.model_name`)
- [ ] Model description and tags set
- [ ] Champion alias assigned after validation
- [ ] Challenger validated against Champion before promotion
- [ ] Promotion gates passed (metrics, latency, bias)

Inference

- [ ] Batch: `fe.score_batch()` tested end-to-end
- [ ] Serving: endpoint created with payload logging
- [ ] Predictions written to Gold layer with `scored_at` timestamp
- [ ] Scheduled retrain job configured (if applicable)

Common Errors

Error	Cause	Solution
`Input X contains NaN`	NaN in feature table	Clean at source with `clean_numeric_features()`
`base_score must be in (0,1)`	Continuous float labels	Binarize to 0/1 integers
`Column not found`	Label in features or missing key	Check `exclude_columns`
`Missing output_schema`	UC registration requirement	Add `output_schema` to `fe.log_model()`
`No model with alias Champion`	Alias not set after registration	`client.set_registered_model_alias()`

Error	Cause	Solution
`Feature table not found`	Wrong catalog/schema	Verify three-level namespace
`Schema mismatch`	Feature table changed after training	Retrain model or pin feature version

---

Related Documents

- GenAI Agent Patterns
- GenAI Standards
- AI Gateway Patterns
- Data Modeling (BI+AI Reuse)
- Gold Layer Patterns

References

- MLflow on Databricks
- Feature Engineering
- UC Model Registry
- Model Serving
- Hyperopt on Databricks
- MLflow Experiment Tracking